

OSRT: Optimized Sparse Recursive Transformer at 600M Scale

Daniel Halwell
CoDHe Labs

June 2026

Abstract

Scaling large language models (LLMs) typically couples physical parameter counts (knowledge capacity) with active computation per token, leading to prohibitive training and inference costs. In this paper, we introduce the **Optimized Sparse Recursive Transformer (OSRT)**, a novel architecture designed to decouple these elements. OSRT-600M contains 601M physical parameters but executes only 278M active parameters per token at inference. It achieves this compute-to-parameter efficiency by combining: (1) a depth-recursive transformer backbone (3 physical blocks recurrently applied 6 times to form 18 effective layers), (2) a hybrid Mixture of Experts (MoE) block with 1 shared and 8 routed experts in a top-2 configuration, (3) Grouped-Query Attention (GQA) with a V-from-K compressed latent KV cache (MLA-lite), and (4) Manifold-Constrained Hyper-Connections (mHC) acting on a 4-channel residual stream. To stabilize the recursive residual stream across 18 unrolled layers, the mHC mixing matrices are projected onto the Birkhoff polytope using log-domain Sinkhorn-Knopp normalization. OSRT-600M is optimized using the second-order Muon optimizer for all 2D weight matrices, yielding identical cross-entropy convergence to first-order optimizers in 52% of the training steps. Finally, we document the empirical lessons learned from the OSRT v5 training run, specifically the identification and mitigation of “loop collapse” via speculative multi-token prediction and loop dropout.

1 Introduction

Modern autoregressive language models have shown extraordinary capabilities as their parameters scale.

However, standard dense transformers require executing the entire parameter footprint for every token. Sparse Mixture of Experts (MoE) architectures, such as DeepSeekMoE [17], partially address this by routing tokens to a subset of experts. In parallel, recursive weight-sharing transformers [2] reuse layer weights to increase the effective depth of the network without multiplying parameter counts.

We present the **Optimized Sparse Recursive Transformer (OSRT)**, a novel architecture that integrates sparse expert mixtures with recursive depth recurrence to achieve high-FLOP efficiency from a compact parameter footprint. OSRT-600M has a physical parameter count of 601M, but requires only 278M active parameters per token. Autoregressive inference computes approximately 2.5B FLOPs-equivalent per token due to a $6\times$ depth recurrence loop.

Crucially, weight reuse across deep unrolled loops introduces numerical instabilities. Small variations in representation magnitude compound exponentially, leading to gradient explosion or representation collapse. To overcome this, OSRT utilizes **Manifold-Constrained Hyper-Connections (mHC)**, which widen the residual stream to 4 independent channels. The channels are combined and scattered back at each sub-layer using dynamic mixing matrices projected onto the Birkhoff polytope of doubly-stochastic matrices. This projects the residual transformation onto a non-expansive manifold, bounding spectral norms and guaranteeing numerical stability across arbitrary recursion depths.

Additionally, OSRT incorporates Hyper-Reparametrized Attention (HRA) adapters on the attention projection paths, a compressed V-from-K latent KV cache (MLA-lite) to reduce the serving memory footprint, and a second-order

optimizer (Muon) for spatial structural parameters. This paper details the mathematical formulation of OSRT, explains the training setup, and shares critical empirical lessons from our development lineage.

2 Related Work

To contextualize OSRT, we review five main pillars: weight-sharing recursion, sparsely-gated MoEs, compressed latent attention, parameter-efficient adapters, and residual/hyper-connections. A comparative summary of OSRT and relevant models is presented in Table 1.

2.1 Weight-Sharing Recursion

Reusing layers recursively along the depth dimension is a classic approach to reduce parameter footprints. Universal Transformers [4] pioneered step-wise recurrence with adaptive computation time. ALBERT [2] demonstrated block sharing for BERT encoders. More recently, Relaxed Recursive Transformers (RRT) [5] introduced depth-wise LoRA relaxations to allow looped layers to specialize. Ouroboros [48] modulates a recurrent backbone using an input-conditioned controller hypernetwork. ModernALBERT [49] integrates cross-layer parameter sharing in an encoder-only setting. Theoretical work by Giannou et al. [6, 51] proves looped transformers can simulate iterative algorithms and execute complex reasoning sequences, motivating the recursion-for-reasoning premise.

2.2 Sparsely-Gated Mixture of Experts

Sparse MoEs decouple parameters from compute by routing tokens. Shazeer et al. [14] introduced Sparsely-Gated MoE layers. GShard [15] and Switch Transformers [16] scaled this to giant networks using top-1/top-2 routing. Mixtral 8×7B [?] demonstrated high-quality open sparse models. DeepSeek-MoE [17] introduced shared experts and fine-grained experts to reduce routing redundancy. OSRT adopts a similar 1 shared + 8 routed experts configuration, relying on dropless grouped-GEMM dispatch (MegaBlocks [20]) and auxiliary-loss-free bias balancing [18, 55].

2.3 Compressed Latent Attention

To combat the KV-cache bottleneck in autoregressive decoding, Multi-Query Attention (MQA) [7] and Grouped-Query Attention (GQA) [8] share KV heads. DeepSeek-V2 [9] introduced Multi-head Latent Attention (MLA), projecting KV states into a low-rank latent vector to compress cache sizes. TransMLA [53] studied post-training translation of GQA structures to MLA. OSRT utilizes an aggressive “V-from-K” latent attention (MLA-lite) that caches only the un-rotated latent vector directly as the Key, deriving the Value via a learned linear map of the cached Key, maximizing memory bandwidth. Long-context attention stability is supported by GQA optimizations [54] and gated attention sinks [?, 52].

2.4 Parameter-Efficient Adapters

Low-Rank Adaptation (LoRA) [?] and Weight-Decomposed LoRA (DoRA) [?] adapt pre-trained parameters using low-rank updates. Householder Reflection Adaptation (HRA) [?, 24] multiplies frozen weights by orthogonal matrices constructed through a chain of learnable Householder reflections. Other orthogonal fine-tuning methods such as HOFT [60] and HTA [59] also seek to preserve pre-trained relational structures. OSRT uses step-specific, rank-256 HRA matrices to parameterize recursive layers, preserving training stability over recursive steps.

2.5 Residual and Hyper-Connections

Deep Residual Learning (ResNet) [69] introduced identity mappings to train deep networks. Hyper-Connections (HC) [21] generalize residual connections by mixing representations across parallel streams using learnable matrices. Manifold-Constrained Hyper-Connections (mHC) [56] constrain multi-stream residual mixing to the Birkhoff polytope via Sinkhorn projections. Spectral-Sphere-Constrained Hyper-Connections (sHC) [57] relax this to a spectral-sphere constraint. mHC-lite [58] avoids iterations by structuring the mixing matrix as a convex combination of permutation matrices. OSRT utilizes the exact log-domain Sinkhorn-Knopp algorithm [22, 23] to enforce doubly stochastic constraints.

Model Name	Total Params	Active Params	Topology	Attention Type	Routing Strategy
OSRT (Ours)	601M	278M	3 blocks $\times$ 6 loops unrolled; per-step HRA (rank 256)	GQA (24/8 heads), MLA latent (“V-from-K”), QK-Norm	Top-2 Sqrt-Softplus gating; 1 shared expert; ALF-LB bias balancing
RingFormer [5]	300M	300M	Recursive loop with static per-step LoRA adapters	Standard MHA latent prepression)	Dense Forward (no sparse routing) Feed-Network
Ouroboros [48]	400M	400M	Recursive loop with dynamic hypernetwork LoRA	Standard MHA latent prepression)	Dense Forward (no sparse routing) Feed-Network
ModernALBERT [49]	120M	120M	Recursive loop with Mixture of LoRAs (MoL)	Multi-Head Attention (MHA)	Low-rank routing (MoL) in FFN path
OLMoE-1B-7B [44]	6.9B	1.3B	16 physical layers (fully parameterized)	MHA (no latent compression)	Top-2 Softmax; 64 routed experts; Switch load balancing
JetMoE-8B [45]	8.0B	2.2B	24 physical layers (fully parameterized)	LLaMA-style GQA (dense)	Top-2 Softmax; 8 routed experts; standard auxiliary loss

Table 1: Systematic comparison of OSRT against leading weight-shared recursive, sparse MoE, and dense baselines.

3 OSRT Architecture

OSRT represents a restructuring of the standard decoder-only transformer. A high-level schematic of the token processing pipeline is provided in Figure 1.

3.1 Parameter Budget

Following the design principles demonstrated in Liquid AI’s LFM2-700M [3], OSRT prioritizes parameter allocation to representation layers over vocabulary embedding layers. In a dense model with a large vocabulary, up to 60% of the parameters can be locked in non-computation embeddings. OSRT balances this ratio at 16.7% embedding overhead for a 65,536 vocabulary. Table 2 lists the exact parameter accounting of the canonical OSRT_605M_A288M preset.

3.2 Depth-Recursive Backbone and Loop Embeddings

OSRT uses 3 physical decoder blocks unrolled 6 times to achieve 18 effective layers. To allow the shared weights to adapt to different depths, we introduce a **loop embedding** $\mathbf{e}_l \in \mathbb{R}^D$ at each loop step $l \in [0, 5]$:

$$\mathbf{x}_{l,b+1} = \text{RecursiveBlock}_b(\mathbf{x}_{l,b}) \quad (1)$$

The loop embedding is added to the router’s input path inside each MoE layer:

$$\mathbf{z}_{\text{router}} = \mathbf{x} + \mathbf{e}_l \quad (2)$$

Symmetry is broken across early layers using loop-indexed deterministic hash routing:

$$\text{expert_id} = (\text{token_id} + l) \pmod{E} \quad (3)$$

This ensures that early routing structures differentiate dynamically before the learned router takes over.

Component	Physical Params	Active/Token
Embedding (tied)	100,690,944	100,690,944
Attention $\times 3$ blocks	17,308,032	17,308,032
mHC Mixers	921,766	921,766
Shared Experts $\times 3$	38,928,384	38,928,384
Routed Experts (3×8)	424,673,280	106,168,320
HRA Adapters ($r = 256$)	14,155,776	14,155,776
MTP Heads $\times 2$ (train)	4,721,664	0
Misc. (Norms, routers, etc.)	45,857	45,857
Total	601,444,393	278,217,769

Table 2: OSRT-600M Parameter Accounting. Naming refers to the canonical code preset; active params exclude speculative Multi-Token Prediction (MTP) heads.

OSRT Autoregressive Block Flow (1 loop iteration):

1. Residual Stream Input: $X \in \mathbb{R}^{4 \times 1536}$
2. Attention Sub-Block:
 - a. Collapse: $z = A_{\text{attn}} \cdot X \in \mathbb{R}^{1536}$ (Sigmoid A_{attn})
 - b. Compute: $f_{\text{attn}}(z)$ (GQA, RoPE, Latent Cache, +HRA)
 - c. Update: $X \leftarrow B_{\text{attn}} \cdot X + C_{\text{attn}} \otimes f_{\text{attn}}(z)$ (Sinkhorn B_{attn})
3. MoE Sub-Block:
 - a. Collapse: $z = A_{\text{ffn}} \cdot X \in \mathbb{R}^{1536}$ (Sigmoid A_{ffn})
 - b. Compute: $f_{\text{moe}}(z)$ (1 shared + 8 routed experts, top-2)
 - c. Update: $X \leftarrow B_{\text{ffn}} \cdot X + C_{\text{ffn}} \otimes f_{\text{moe}}(z)$ (Sinkhorn B_{ffn})
4. Loop back (6 iterations total) with per-loop embedding.

Figure 1: Autoregressive flow of the OSRT block.

3.3 GQA + V-from-K Latent KV Cache (MLA-lite)

Autoregressive generation is bottlenecked by Memory Bandwidth due to the loading of Key-Value (KV) caches. OSRT addresses this with a compressed latent KV cache mechanism inspired by DeepSeek’s Multi-head Latent Attention (MLA).

Instead of caching projection matrices $K, V \in \mathbb{R}^{B \times S \times H \times D_h}$ directly, the attention block projects input tokens to a low-dimensional latent vector $\mathbf{c}_{\text{kv}} \in \mathbb{R}^{512}$:

$$\mathbf{c}_{\text{kv}} = \mathbf{x} W_{\text{kv_down}} \quad (4)$$

where $W_{\text{kv_down}} \in \mathbb{R}^{1536 \times 512}$. During generation, only $\mathbf{c}_{\text{kv}}$ is stored in the KV cache, requiring 512 scalar allocations per token instead of $8 \times 64 \times 2 = 1024$ scalars under a Grouped-Query Attention

(GQA) baseline.

At attention computation, Keys (K) and Values (V) are reconstructed from $\mathbf{c}_{\text{kv}}$:

$$K = \text{view}(\mathbf{c}_{\text{kv}}) \cdot W_{K_proj} \quad (5)$$

$$V = \mathbf{c}_{\text{kv}} W_{V_from_k} \quad (6)$$

where $W_{V_from_k} \in \mathbb{R}^{512 \times 512}$ with a bias vector. The queries Q are projected from the hidden state.

To prevent attention scale explosion during early optimization steps, we apply QK-Norm [12], normalizing Q and K along the head dimension before computing dot products:

$$\tilde{Q} = \text{RMSNorm}(Q), \quad \tilde{K} = \text{RMSNorm}(K) \quad (7)$$

Positional encoding is injected via Rotary Position Embeddings (RoPE) [11] applied to the last 64 dimensions of the normalized Q and K . Exact attention is computed using the FlashAttention-3 path [13] for speed.

3.4 Mixture of Experts and Grouped-GEMM

Each physical block contains an MoE block with 1 shared expert ($h_{\text{shared}} = 2816$) and 8 routed experts ($h_{\text{routed}} = 3840$). The routing follows a hybrid top-2 configuration. Given a token representation $\mathbf{z}$, the router computes affinities $\mathbf{s} \in \mathbb{R}^8$ using a $\sqrt{\text{Softplus}}$ activation:

$$s_i = \sqrt{\text{Softplus}(\mathbf{z} \mathbf{w}_{\text{route}, i} + b_i)} \quad (8)$$

The gating scores are normalized over the selected top-2 experts:

$$g_i = \frac{s_i}{\sum_{j \in \text{top-2}} s_j} \quad (9)$$

OSRT implements a non-blocking **Grouped-GEMM dispatch** [20]. Tokens are flattened, sorted by expert indices, and processed in a single kernel launch using `torch._grouped_mm`. This enables compiling the entire MoE block into a single CUDA graph. Furthermore, we apply a stability clamp on the expert SwiGLU blocks:

$$\text{SwiGLU}(\mathbf{x}) = W_{\text{down}}(\text{SiLU}(G(\mathbf{x})) \odot U(\mathbf{x})) \quad (10)$$

$$G(\mathbf{x}) = \text{clamp}(\mathbf{x}W_{\text{gate}}, -\infty, 10.0) \quad (11)$$

$$U(\mathbf{x}) = \text{clamp}(\mathbf{x}W_{\text{up}}, -10.0, 10.0) \quad (12)$$

3.5 Manifold-Constrained Hyper-Connections

Reusing weight matrices 6 times in a loop creates a multiplicative feedback loop. Any spectral radius of the residual block transform greater than 1 results in exponential growth of representations. OSRT resolves this instability using Manifold-Constrained Hyper-Connections (mHC), modifying the multi-stream residual concept of Zhu et al. [21]. The residual stream is widened to a multi-channel tensor $X \in \mathbb{R}^{4 \times 1536}$.

For each sub-block F (attention or MoE), mHC generates three per-token matrices dynamically from the normalized representation of X :

$$\mathbf{a} = \sigma(\alpha_{\text{pre}}XW_{\text{pre}} + s_{\text{pre}}) \in \mathbb{R}^{1 \times 4} \quad (13)$$

$$\mathbf{c} = 2 \cdot \sigma(\alpha_{\text{post}}XW_{\text{post}} + s_{\text{post}}) \in \mathbb{R}^{4 \times 1} \quad (14)$$

$$B_{\text{raw}} = \alpha_{\text{res}}XW_{\text{res}} + s_{\text{res}} \in \mathbb{R}^{4 \times 4} \quad (15)$$

The residual mixing matrix $B \in \mathbb{R}^{4 \times 4}$ is projected onto the Birkhoff polytope of doubly-stochastic matrices using log-domain Sinkhorn-Knopp normalization [22, 23] (20 iterations):

$$B = \text{Sinkhorn}(B_{\text{raw}}) \quad (16)$$

Log-domain normalization alternates row and column log-sum-exp subtractions to preserve gradients and prevent numerical underflow/overflow:

$$\log M_{i,j}^{(t+1/2)} = \log M_{i,j}^{(t)} - \log \sum_k M_{i,k}^{(t)} \quad (17)$$

$$\log M_{i,j}^{(t+1)} = \log M_{i,j}^{(t+1/2)} - \log \sum_k M_{k,j}^{(t+1/2)} \quad (18)$$

The doubly-stochastic constraint ensures that rows and columns of B sum to 1. By the Birkhoff-von

Neumann theorem, B lies in the convex hull of permutation matrices. Consequently, its spectral norm is bounded:

$$\|B\|_2 \leq 1 \quad (19)$$

The update rule is formulated as:

$$X_{\text{next}} = B \cdot X + \mathbf{c} \otimes F(\mathbf{a} \cdot X) \quad (20)$$

Because $\|B\|_2 \leq 1$, the residual transform is strictly non-expansive, conserving spectral energy and preventing representation decay across 18 unrolled layers.

3.6 Householder Reflection Adapters (HRA)

To allow step-wise specialization along the recurrent stack without multiplying the weight parameter footprint, OSRT introduces step-specific rank-256 HRA adapters [?, 24] on the tied attention projections:

$$W_{\text{adapted}} = W_{\text{frozen}} \cdot R_{\text{HRA}} \quad (21)$$

where R_{HRA} is constructed orthogonal matrix computed through a sequence of learnable Householder reflections:

$$R_{\text{HRA}} = \prod_{k=1}^r (I - 2\mathbf{v}_k\mathbf{v}_k^T) \quad (22)$$

This orthogonal parameterization prevents representational drift and stabilizes training.

3.7 Multi-Token Prediction Speculation

To reduce inference latency, OSRT adds two Speculative Multi-Token Prediction (MTP) heads [10, 25] during training. The primary LM head processes the representation from loop step 6 ($l = 6$) to predict token t_{n+1} . The two MTP heads consume intermediate states at $l = 4$ and $l = 5$ to predict t_{n+2} and t_{n+3} respectively. During inference, these training-only heads are discarded.

4 Pretraining and Post-Training Pipeline

4.1 Tokenization

OSRT implements a 100,000 token vocabulary using byte-level Byte-Pair Encoding (BPE) [26, 27]. Crucially, we enforce digit isolation, tokenizing individ-

ual digits separately. This prevents numerical fragmentation, preserving the alignment of numerical values in mathematical contexts.

4.2 Pretraining Data and Curriculum

The model was pretrained on 1.2 Trillion tokens. The pretraining corpus represents a math-first data mixture:

- **40% FineWeb-Edu** [28]: High-quality educational text extracted from Web datasets.
- **30% Nemotron-CC-Math** [29]: Mathematical and scientific Common Crawl text.
- **20% Cosmopedia** [30, 31]: Synthetic textbooks and course materials.
- **10% CodeRepos**: Open-source software source code.

OSRT uses a progressive sequence-length curriculum (SGDR [32]) over three phases: 60% of training at 2048 context length, 30% at 4048, and the remaining 10% at 8192 context length. Under a Chinchilla-optimal setup [33], this guarantees sample diversity.

4.3 Post-Training and Reinforcement Learning

Post-training begins with a Supervised Fine-Tuning (SFT) stage [34] using multi-teacher on-policy distillation [35] on mathematical reasoning and programming tasks. Following SFT, we apply Group Relative Policy Optimization (GRPO) [36] for reinforcement learning. To prevent reward hacking, GRPO relies entirely on verifiable rewards (RLVR [37, 62]): python code execution compilers and LaTeX-equivalence parsers, augmented by Difficulty-Aware Group Policy Optimization (DGPO) [61] to stabilize rewards on hard proofs.

5 Optimizer and Training Configuration

5.1 Muon Optimizer

Standard first-order optimizers (AdamW [38], Lion) update parameters coordinates-wise. For 2D matrices, this ignores spatial correlation. OSRT utilizes the second-order **Muon** optimizer [39, 40] (descended from Shampoo [41]) for all 2D weight matrices. Muon tracks SGD-momentum and projects

the update onto the nearest semi-orthogonal matrix using a Newton-Schulz polynomial iteration [63]. Given update momentum $G \in \mathbb{R}^{M \times N}$, the orthogonalized update X is computed via a 5-step quintic polynomial recursion:

$$X_0 = \frac{G}{\|G\|_F} \quad (23)$$

$$X_{k+1} = c_1 X_k + c_2 (X_k X_k^T) X_k + c_3 (X_k X_k^T)^2 X_k \quad (24)$$

where $c_1 = 3.4445$, $c_2 = -4.7750$, $c_3 = 2.0315$. By enforcing update orthogonality, Muon equalizes the singular spectrum. All 1D parameters (biases, RMSNorm scales, and embeddings) are optimized using standard AdamW. Memory efficiency is maximized during training using Liger kernels [42] and Cut Cross-Entropy [43].

5.2 Auxiliary-Loss-Free Expert Balancing

To maintain balance across the 8 routed experts without corrupting the language-model gradients, we implement the DeepSeek-V3 auxiliary-loss-free balancing scheme [18, 55]. Rather than adding a routing entropy term to the loss function, the router dynamically adjusts the expert selection bias b_i :

$$b_i \leftarrow b_i + \gamma \cdot (\text{target_load} - \text{observed_load}_i) \quad (25)$$

where $\gamma = 0.001$. This optimization is performed out-of-graph, ensuring that the gradient pathways remain focused entirely on the primary cross-entropy objective. We also apply a router z-loss [19] for numerical stability.

6 Empirical Results and Lessons Learned

6.1 Mitigating Loop Collapse

During the development of nano-osrt v5 (363M parameters), post-training evaluations using layer ablation revealed a severe failure mode: **loop collapse**. We observed that the final loop iteration ($l = 5$) was performing approximately 90% of the cross-entropy reduction, while intermediate loops ($l \in [0, 4]$) contributed negligible representational progress. The model had collapsed into a shallow network, wasting compute on earlier loops.

To force intermediate loops to build useful representations, we implemented two mechanisms:

1. **Per-Loop Auxiliary Loss:** We applied the cross-entropy loss function to the intermediate outputs of all loops during training, weighted by $\alpha_{\text{aux}} = 0.05$.
2. **Loop Dropout:** During pretraining, we randomly truncated the unrolled loop depth to a random integer $K \in [3, 6]$ with a probability of 10%.

Following these fixes, cross-entropy evaluations showed monotonic, balanced progress across all 6 loops, proving that loop collapse was successfully resolved. We monitor loop collapse using relative residual update telemetry:

$$\delta_l = \frac{\|\mathbf{x}_l - \mathbf{x}_{l-1}\|_2}{\|\mathbf{x}_{l-1}\|_2} \quad (26)$$

6.2 Optimization Efficiency

Pretraining runs comparing Muon to the Lion optimizer on the OSRT architecture demonstrated a massive sample efficiency gap. Under identical training data mixtures and context windows, Muon achieved a final cross-entropy loss of **3.43** within 1200 steps, whereas the Lion baseline achieved only **7.4**. This represents a 4-point cross-entropy advantage, which translates to a $> 40\%$ saving in training FLOPs.

7 Conclusion

We have presented OSRT, an architecture that combines depth-recursive transformers, hybrid Mixture of Experts, and Manifold-Constrained HyperConnections. By projecting residual mixing matrices onto the Birkhoff polytope via log-domain Sinkhorn normalization, OSRT guarantees numerical stability across deep recurrent stacks. Optimized via Muon and speculative Multi-Token Prediction, OSRT-600M provides a viable framework for high-efficiency, memory-bandwidth-friendly LLM deployment.

References

- [1] DeepSeek-AI. (2024). DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts. arXiv preprint arXiv:2401.06066.
- [2] Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P., & Soricut, R. (2019). ALBERT: A Light BERT for Self-supervised Learning of Language Representations. arXiv preprint arXiv:1909.11942.
- [3] Liquid AI. (2025). LFM2-700M: High-Efficiency Parameter Allocations for Small Language Models. Technical Report.
- [4] Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., & Kaiser, Ł. (2018). Universal Transformers. arXiv preprint arXiv:1807.03819.
- [5] Bae, S. et al. (2024). Relaxed Recursive Transformers: Effective Parameter Sharing with Layer-wise LoRA. arXiv preprint arXiv:2410.20672.
- [6] Giannou, A. et al. (2023). Looped Transformers as Programmable Computers. arXiv preprint arXiv:2301.13196.
- [7] Shazeer, N. (2019). Fast Transformer Decoding: One Write-Head is All You Need. arXiv preprint arXiv:1911.02150.
- [8] Ainslie, J. et al. (2023). GQA: Training Generalized Multi-Query Transformer Models. arXiv preprint arXiv:2305.13245.
- [9] DeepSeek-AI. (2024). DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. arXiv preprint arXiv:2405.04434.
- [10] DeepSeek-AI. (2024). DeepSeek-V3 Technical Report. arXiv preprint arXiv:2412.19437.
- [11] Su, J. et al. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv preprint arXiv:2104.09864.
- [12] Henry, A. et al. (2020). Query-Key Normalization for Transformers. arXiv preprint arXiv:2010.04245.
- [13] Shah, J. et al. (2024). FlashAttention-3: Fast and Accurate Attention with Asynchrony. arXiv preprint arXiv:2407.08608.
- [14] Shazeer, N. et al. (2017). Outrageously Large Neural Networks: Sparsely-Gated MoE. arXiv preprint arXiv:1701.06538.

- [15] Lepikhin, D. et al. (2020). GShard: Scaling Giant Models with Conditional Computation. arXiv preprint arXiv:2006.16668.
- [16] Fedus, W., Zoph, B., & Shazeer, N. (2021). Switch Transformers: Scaling to Trillion Parameter Models. arXiv preprint arXiv:2101.03961.
- [17] Dai, D. et al. (2024). DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts. arXiv preprint arXiv:2401.06066.
- [18] Wang, L. et al. (2024). Auxiliary-Loss-Free Load Balancing Strategy for Mixture-of-Experts. arXiv preprint arXiv:2408.15664.
- [19] Zoph, B. et al. (2022). ST-MoE: Designing Stable and Transferable Sparse Expert Models. arXiv preprint arXiv:2202.08906.
- [20] Gale, T. et al. (2022). MegaBlocks: Efficient Sparse Training with Mixture-of-Experts. arXiv preprint arXiv:2211.15841.
- [21] Zhu, D. et al. (2024). Hyper-Connections. arXiv preprint arXiv:2409.19606.
- [22] Cuturi, M. (2013). Sinkhorn Distances: Light-speed Computation of Optimal Transport. In NeurIPS 2013.
- [23] Sinkhorn, R., & Knopp, P. (1967). Concerning Nonnegative Matrices and Doubly Stochastic Matrices. *Pacific Journal of Mathematics*, 21(2), 343-348.
- [24] Yuan, S., Liu, H., & Xu, H. (2024). Bridging the Gap between Low-rank and Orthogonal Adaptation via HRA. arXiv preprint arXiv:2405.17484.
- [25] Gloeckle, F. et al. (2024). Better & Faster Large Language Models via Multi-token Prediction. arXiv preprint arXiv:2404.19737.
- [26] Sennrich, R., Haddow, B., & Birch, A. (2015). Neural Machine Translation of Rare Words with Subword Units. arXiv preprint arXiv:1508.07909.
- [27] Radford, A. et al. (2019). Language Models are Unsupervised Multitask Learners. OpenAI Technical Report.
- [28] Penedo, G. et al. (2024). The FineWeb Datasets: Decanting the Web for Text Data at Scale. arXiv preprint arXiv:2406.17557.
- [29] Karimi Mahabadi, R. et al. (2025). Nemotron-CC-Math: A 133 Billion-Token Math Pretraining Dataset. arXiv preprint arXiv:2508.15096.
- [30] Ben Allal, L. et al. (2024). Cosmopedia. Hugging Face Blog.
- [31] Gunasekar, S. et al. (2023). Textbooks Are All You Need. arXiv preprint arXiv:2306.11644.
- [32] Loshchilov, I., & Hutter, F. (2016). SGDR: Stochastic Gradient Descent with Warm Restarts. arXiv preprint arXiv:1608.03983.
- [33] Hoffmann, J. et al. (2022). Training Compute-Optimal Large Language Models. arXiv preprint arXiv:2203.15556.
- [34] Ouyang, L. et al. (2022). Training Language Models to Follow Instructions with Human Feedback. In NeurIPS 2022.
- [35] Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the Knowledge in a Neural Network. arXiv preprint arXiv:1503.02531.
- [36] Shao, Z. et al. (2024). DeepSeekMath: Pushing the Limits of Mathematical Reasoning. arXiv preprint arXiv:2402.03300.
- [37] Lambert, N. et al. (2024). Tülu 3: Pushing Frontiers in Open Language Model Post-Training. arXiv preprint arXiv:2411.15124.
- [38] Loshchilov, I., & Hutter, F. (2017). Decoupled Weight Decay Regularization. arXiv preprint arXiv:1711.05101.
- [39] Jordan, K. (2024). Muon: An Optimizer for Hidden Layers. Blog Post.
- [40] Liu, J. et al. (2025). Muon is Scalable for LLM Training. arXiv preprint arXiv:2502.16982.
- [41] Gupta, V., Koren, T., & Singer, Y. (2018). Shampoo: Preconditioned Stochastic Tensor Optimization. arXiv preprint arXiv:1802.09568.
- [42] Hsu, P. et al. (2024). Liger Kernel: Efficient Triton Kernels for LLM Training. arXiv preprint arXiv:2410.10989.
- [43] Wijmans, E. et al. (2024). Cut Your Losses in Large-Vocabulary Language Models. arXiv preprint arXiv:2411.09009.

- [44] Muennighoff, N. et al. (2024). OLMoE: Open Mixture-of-Experts Language Models. arXiv preprint arXiv:2409.02060.
- [45] Shen, Y. et al. (2024). JetMoE: Reaching Llama2 Performance with 0.1M Dollars. arXiv preprint arXiv:2404.07413.
- [46] Hu, S. et al. (2024). MiniCPM: Unveiling the Potential of Small Language Models. arXiv preprint arXiv:2404.06395.
- [47] Abdin, M. et al. (2024). Phi-3 Technical Report. arXiv preprint arXiv:2404.14219.
- [48] Bae, S. et al. (2026). Ouroboros: Gated Recurrence with Input-Conditioned LoRA for Recursive Transformers. arXiv preprint arXiv:2604.02051.
- [49] Bae, S. et al. (2025). ModernALBERT. arXiv preprint arXiv:2512.12880.
- [50] Bae, S. et al. (2025). Mixture of Recursions. arXiv preprint arXiv:2507.10524.
- [51] Giannou, A. et al. (2025). Reasoning with Latent Thoughts: On the Power of Looped Transformers. arXiv preprint arXiv:2502.17416.
- [52] Qiu, J. et al. (2025). Gated Attention Sinks: Eliminating Sinks, Bounding Activations, and Scaling Context. In NeurIPS 2025.
- [53] Chen, Y. et al. (2025). TransMLA: Translating GQA to MLA. arXiv preprint arXiv:2502.07864.
- [54] Chen, Y. et al. (2025). Cost-Optimal Grouped-Query Attention for Long-Context Modeling. arXiv preprint arXiv:2503.09579.
- [55] Han, X. Y., & Zhong, Y. (2025). A Theoretical Framework for Auxiliary-Loss-Free Load Balancing of Sparse Mixture-of-Experts in Large-Scale AI Models. arXiv preprint arXiv:2512.03915.
- [56] Xie, Z. et al. (2025). mHC: Manifold-Constrained Hyper-Connections. arXiv preprint arXiv:2512.24880.
- [57] Xie, Z. et al. (2026). Beyond the Birkhoff Polytope: Spectral-Sphere-Constrained Hyper-Connections. arXiv preprint arXiv:2603.20896.
- [58] Yang, L. et al. (2026). mHC-lite: You Don't Need 20 Sinkhorn-Knopp Iterations. arXiv preprint arXiv:2601.05732.
- [59] Su, H. et al. (2024). Householder Transformation-based Adaptor. arXiv preprint arXiv:2410.22952.
- [60] Yuan, S. et al. (2025). HOFT: Householder Orthogonal Fine-tuning. arXiv preprint arXiv:2505.16531.
- [61] Anonymous. (2026). DGPO: Difficulty-Aware Group Policy Optimization. arXiv preprint arXiv:2601.20614.
- [62] Anonymous. (2026). LongRLVR: Long-Context RL with Verifiable Rewards. arXiv preprint arXiv:2603.02146.
- [63] Kim, G. Y. et al. (2026). Convergence of Muon with Newton-Schulz. arXiv preprint arXiv:2601.19156.
- [64] Anonymous. (2024). Why Warmup the Learning Rate? Underlying Mechanisms and Empirical Study. arXiv preprint arXiv:2406.09405.
- [65] Anonymous. (2024). Reconciling Kaplan and Chinchilla Scaling Laws. arXiv preprint arXiv:2406.12907.
- [66] Wang, Y. et al. (2025). Ultra-FineWeb: Efficient Data Filtering and Verification for High-Quality LLM Training Data. arXiv preprint arXiv:2505.05427.
- [67] Su, D. et al. (2024). Nemotron-CC: Transforming Common Crawl into a Refined Long-Horizon Pretraining Dataset. arXiv preprint arXiv:2412.02595.
- [68] Li, Y. et al. (2023). Textbooks Are All You Need II: phi-1.5 technical report. arXiv preprint arXiv:2309.05463.
- [69] He, K. et al. (2015). Deep Residual Learning for Image Recognition. In CVPR 2016.
- [70] Hwang, C. et al. (2022). Tutel: Adaptive Mixture-of-Experts at Scale. In MLSys 2023.